

Autonomous RosBot Maze Navigation

Technical Approach and Simulation Timing Report

Summary. Two controllers share the same navigation architecture. The Maze 3 version uses more conservative obstacle and recovery thresholds, while the Maze 4/5 version is tuned for narrow zigzags and floating-wall variability through lighter obstacle blocking, local unhook maneuvers, wide-corner following, traversed-corridor softening, and sticky-escape planning.

1. Measured Mission Timings

The required intervals are measured from simulation start to the blue pillar, then from blue completion to the yellow pillar. Total time is included for completeness.

Environment	Start simulation -> blue reached	Blue reached -> yellow reached	Total mission time
Maze 3	2 min 31 s	1 min 26 s	3 min 57 s
Maze 4	1 min 38 s	1 min 53 s	3 min 31 s
Maze 5	39 s	54 s	1 min 33 s

2. Common Base Used by Both Controllers

The two submissions are not independent solutions. They use the same modular base: `config.py` for tuning, `robot_io.py` for Webots devices, `occupancy_grid.py` for mapping and planning layers, `mission_runtime.py` for the state machine and control loop, and a small `roswbot_controller.py` launcher.

Pose estimation: Wheel encoders estimate translation while the compass/IMU heading constrains orientation. The pose is updated continuously and is used to project LiDAR, depth, green-floor, and target observations into the same global grid.

Evidence-based LiDAR map: Each LiDAR beam decreases occupancy evidence along free cells and increases evidence at the measured endpoint. A cell becomes free or occupied only after repeated evidence, which is more stable than copying a single scan into a map.

Semantic layers: Green floor is stored separately as a hard forbidden layer. The depth camera can also detect low/floating obstacles that a horizontal 2-D LiDAR may miss. Keeping semantic layers separate prevents LiDAR free rays from erasing them.

Frontier exploration: While the required pillar is unknown, the robot selects frontier cells - known free cells touching unknown space - clusters and scores them, then uses A* to explore systematically.

Mission state machine: The shared sequence is SEARCH_BLUE -> NAVIGATE_BLUE -> SEARCH_YELLOW -> NAVIGATE_YELLOW -> FINISHED. Yellow may be detected and cached while blue remains the active target.

Target approach: The robot does not plan to the pillar centre. It samples reachable standoff cells around the RGB-D target estimate, checks line of sight and map clearance, then follows an A* path to an approach point. Final completion is confirmed from camera colour coverage.

Path monitoring and recovery: The active path is continuously checked against new wall, green, and semantic information. If invalid or physically blocked, the controller stops, performs a local recovery, and replans rather than blindly continuing.

3. How the Robot Is Operated

- 1. Sense:** Read encoders, compass/IMU heading, LiDAR, RGB image, depth image, and front/rear IR distances.
- 2. Localize and map:** Update odometry pose; ray-cast LiDAR measurements into the occupancy evidence grid; update green and optional low-clearance semantic layers.
- 3. Detect mission targets:** Segment blue and yellow pixels, obtain valid depth around the detection, estimate global target position, and update the target cache.
- 4. Select a goal:** If the active target is known, select a safe approach cell. Otherwise, select a reachable frontier. In Maze 4/5, a sticky open-space escape goal may be selected after repeated planning failures.
- 5. Plan:** Build the effective blocked-cell set, inflate obstacles for the robot footprint, and run 8-connected A*. Simplify the cell path into waypoints.
- 6. Execute:** Apply heading-proportional wheel control toward each waypoint. The Maze 4/5 version additionally slows in narrow corridors and uses wide-corner/unhook primitives.
- 7. Validate and recover:** Continuously test the remaining path and live sensors. Replan for newly mapped hazards; use local physical recovery when the problem is execution rather than global path validity.
- 8. Finish:** Camera colour coverage confirms blue, switches the state to yellow, and finally confirms yellow before stopping and recording the timing.

4. Design Rationale

The Maze 3 controller is the safer baseline: it assumes close observations are likely to be real hazards. The Maze 4/5 controller keeps the same global intelligence but is deliberately more permissive locally, because false blockage from floating-wall/depth interpretation is more damaging than a short, controlled close-clearance maneuver. The resulting design is hybrid: global A* and frontier exploration provide structure, while local unhook, wide-corner, narrow-mode, and sticky-escape behaviors handle the non-ideal geometry that the grid map cannot represent perfectly.